Jack Fix It — Plano de Desenvolvimento do MVP

Oct 15, 2026 · @cesar

Visão geral

O MVP do Jack Fix It é um marketplace de serviços domésticos para o Sul da Flórida no modelo "Uber de serviços": o cliente pede, um profissional verificado executa e a plataforma segura o pagamento até o serviço ser confirmado. O desenvolvimento cabe em **20 dias úteis de 8 h**, dentro do limite de 30 dias, e o restante fica como folga para a revisão das lojas.

Objetivo do MVP: validar o modelo com jobs reais. O deck define a saída do MVP 1 com 100+ jobs concluídos, conclusão ≥ 90%, nota média ≥ 4,7 e disputas abaixo de 2%, então o sistema mede isso desde o primeiro dia.

O que o produto precisa entregar:

- **Para o cliente:** profissional verificado, preço fechado antes do serviço, pagamento protegido e suporte se algo der errado.
- **Para o profissional:** nada de pagar por lead; ele recebe só quando o job acontece, com clientes vindo pela plataforma.
- **Para a operação (BDC):** um painel para responder leads em menos de 5 minutos, fazer o match e acompanhar cada job até o fim.

Premissas deste plano: um desenvolvedor, dias úteis de segunda a sexta, marca e textos legais fornecidos pelos sócios. O plano conta só tempo de execução, sem custos.

Decisões fechadas: marca Jack Fix It; cores azul-marinho, laranja, branco e cinza claro; app mobile em React Native (cliente e profissional) com Stripe já no MVP; lançamento com as 5 categorias do deck. O README do repositório antigo (Lovable) serve só como referência de conteúdo e UX.

Decisão pendente: hospedagem. As opções são a VPS da Hostinger (já existe, custo quase zero, segurança e operação por nossa conta) ou o Railway Pro (custo mensal, com backups com PITR, SOC 2, audit log e rede privada prontos). Ambas rodam nos EUA, sem diferença de latência. Custos e prós e contras estão no doc de custos; este plano funciona com as duas.

Escopo do MVP

Entra o ciclo completo de um job: pedido → orçamento → pagamento retido → execução → confirmação → repasse → avaliação. Fica fora tudo que não é necessário para os primeiros 100 jobs.

Dentro do MVP

Módulo	O que entrega
Autenticação	Cadastro e login por e-mail e senha, verificação de telefone por SMS (OTP), recuperação de senha, exclusão de conta
App do cliente	Escolher categoria, criar pedido com fotos e endereço, comparar orçamentos, pagar, acompanhar o job, chat, confirmar, avaliar, histórico
App do profissional	Onboarding, envio de documentos, conta de recebimento (Stripe), feed de pedidos, orçamento, agenda, status do job, fotos de antes e depois, ganhos
Pagamento protegido	Cobrança no aceite do orçamento, valor retido na plataforma, repasse após confirmação, take rate configurável, reembolso e disputa
Confiança	Selo de verificado, avaliações dos dois lados, perfil público do profissional, janela de garantia para abrir problema
Comunicação	Chat por job, push, e-mail e SMS transacionais
Painel admin/BDC (web)	Fila de leads com cronômetro, aprovação de profissionais, gestão de jobs, match manual, liberação e estorno, disputas, KPIs
Landing page	Página de produto mobile-first com o fluxo "conte o que aconteceu", páginas de serviço para SEO e captura de UTM
Idiomas	Inglês e espanhol (português opcional)

Categorias iniciais

Limpeza (house cleaning), Handyman (marido de aluguel), Pintura, Jardinagem e Montagem/Mudança, as 5 do deck. As categorias são cadastradas no admin, então AC, encanamento, elétrica, piscina e eletrodomésticos entram depois sem mexer no código; as que exigem licença na Flórida só abrem quando a verificação de licença estiver validada.

Fora do MVP

- Rastreamento do profissional em tempo real no mapa (o MVP tem status "a caminho" e check-in com localização)
- Background check automático (Checkr): no MVP a verificação é manual pelo admin
- Análise de ligações com IA, CRM próprio, planos recorrentes, contas B2B
- Pagamento por marcos em jobs grandes, gorjeta, cupons e indicação
- Mascaramento de telefone e chamadas pelo app

Perfis e fluxos principais

Cliente e profissional usam o mesmo app React Native, com o modo escolhido no cadastro; admin/BDC usa o painel web.

Fluxo do cliente

1. Cria conta e verifica o telefone.
2. Conta o que aconteceu (texto, foto ou vídeo) ou escolhe uma categoria, responde perguntas de contexto e informa endereço e data.
3. Recebe orçamentos de profissionais verificados (ou um match feito pelo BDC) e compara preço, nota e data.
4. Aceita um orçamento e paga pelo app. O valor fica retido.
5. Acompanha o status: agendado, a caminho, em andamento, concluído.
6. Confirma a conclusão ou abre um problema dentro da janela de garantia.
7. Avalia o profissional.

Fluxo do profissional

1. Cria conta e preenche perfil, categorias e área de atuação (ZIP codes).
2. Envia documento com foto, licença (quando a categoria exige) e seguro.
3. Conecta a conta de recebimento pelo onboarding do Stripe.

4. Aguarda a aprovação do admin e recebe o selo de verificado.
5. Vê pedidos compatíveis e envia orçamento com preço fechado, escopo e data.
6. No dia, marca "a caminho", faz check-in, envia fotos de antes e depois e conclui.
7. Recebe o repasse após a confirmação e avalia o cliente.

Fluxo do admin/BDC

1. Novo lead (app ou landing) entra na fila com cronômetro; a meta de primeiro contato é menos de 5 minutos.
2. Qualifica o pedido e, se nenhum profissional responder, faz o match manual.
3. Aprova ou recusa profissionais depois de conferir os documentos.
4. Acompanha jobs, resolve disputas e libera ou estorna pagamentos.
5. Acompanha os KPIs no dashboard.

Regras de negócio

- Só profissionais aprovados e com Stripe ativo veem pedidos.
- Cada pedido vai para até N profissionais da categoria e da área (N configurável, padrão 5).
- Orçamento expira se não for aceito em X horas (configurável).
- Se o cliente não confirmar nem abrir problema em 48 h após a conclusão, o job é confirmado automaticamente.
- Cancelamento grátis até 24 h antes; depois disso, a regra é definida pelos sócios.
- Telefones, e-mails e links escritos no chat são ocultados, para manter o pagamento dentro do app.

Status que o cliente vê

Status	Quando aparece
Request Received	Pedido criado
Reviewing	BDC está classificando ou pedindo mais informação
Finding a Pro	Pedido enviado aos profissionais
Quote Ready	Há orçamento para aprovar

Status	Quando aparece
Scheduled	Pago e com data marcada
Professional On The Way	Profissional marcou "a caminho"
In Progress	Check-in feito
Waiting for Confirmation	Profissional concluiu; janela de 48 h aberta
Completed	Confirmado e repassado
Needs Attention	Problema aberto ou ação pendente do cliente
Cancelled	Cancelado por qualquer lado

Landing page (página de produto)

A landing é uma página de produto mobile-first com um único trabalho: fazer o cliente abrir um pedido em cerca de 60 segundos. O conteúdo vem do README do repositório antigo (Lovable); a marca é Jack Fix It, em azul-marinho, laranja e branco.

Estrutura da página

1. **Header:** logo, Serviços, Como funciona, Para profissionais, Entrar e o CTA "Tell Jack".
2. **Hero:** "Whatever happens at home, we fix it." com uma linha de apoio; CTA principal "Tell Jack What Happened" e secundário "Explore Services"; selos curtos de profissionais verificados, agendamento fácil, pagamento seguro e suporte real.
3. **"What's going on at home?":** o bloco principal. Campo de texto ("I need a ceiling fan installed…"), upload de foto e de vídeo; abre o fluxo de pedido sem exigir login.
4. **Como funciona:** Tell Jack → We find the right pro → Get it done → Support when you need it.
5. **Serviços:** grade de categorias para quem prefere navegar, sem dominar a página.
6. **Confiança:** não precisa saber para quem ligar, tudo organizado num lugar, pessoas reais quando precisar, histórico do serviço guardado.
7. **Avaliações:** só entram quando houver avaliações reais; nenhum depoimento inventado.
8. **Para profissionais:** sem pagar por lead, recebe quando o job acontece, link para o app.
9. **Rodapé:** Serviços, Para profissionais, Áreas atendidas, Ajuda, Termos, Privacidade, Contato.

Fluxo de pedido pela web

1. **O que aconteceu:** texto, fotos e vídeo; categoria opcional.
2. **Perguntas de contexto** por categoria, cadastradas no admin. Exemplos: limpeza (quartos, banheiros, metragem, única ou recorrente), handyman (vários pequenos serviços num pedido só).
3. **Local:** ZIP, endereço, data e horário preferidos. ZIP fora da área vai para lista de espera.
4. **Contato:** nome, sobrenome, telefone, e-mail e forma preferida (texto, ligação ou e-mail).
5. **Confirmação:** "Jack is on it", com número do pedido (ex.: JF-000124) e convite para criar conta e acompanhar.

Pedido sem categoria cai na fila do BDC, que classifica antes de distribuir. Voz não aparece até funcionar: nada que não funciona é mostrado como se funcionasse.

Regras de texto

Até haver aprovação jurídica, landing e app não prometem seguro, garantia de mão de obra, escrow, licença em todas as categorias nem reembolso garantido. O texto para o cliente usa formulações como "Support when you need it", "Verified information where applicable", "Clear service records" e "Secure payment experience". Neste doc, "pagamento protegido" e "garantia" são nomes internos do fluxo, não texto de marketing.

SEO e áreas atendidas

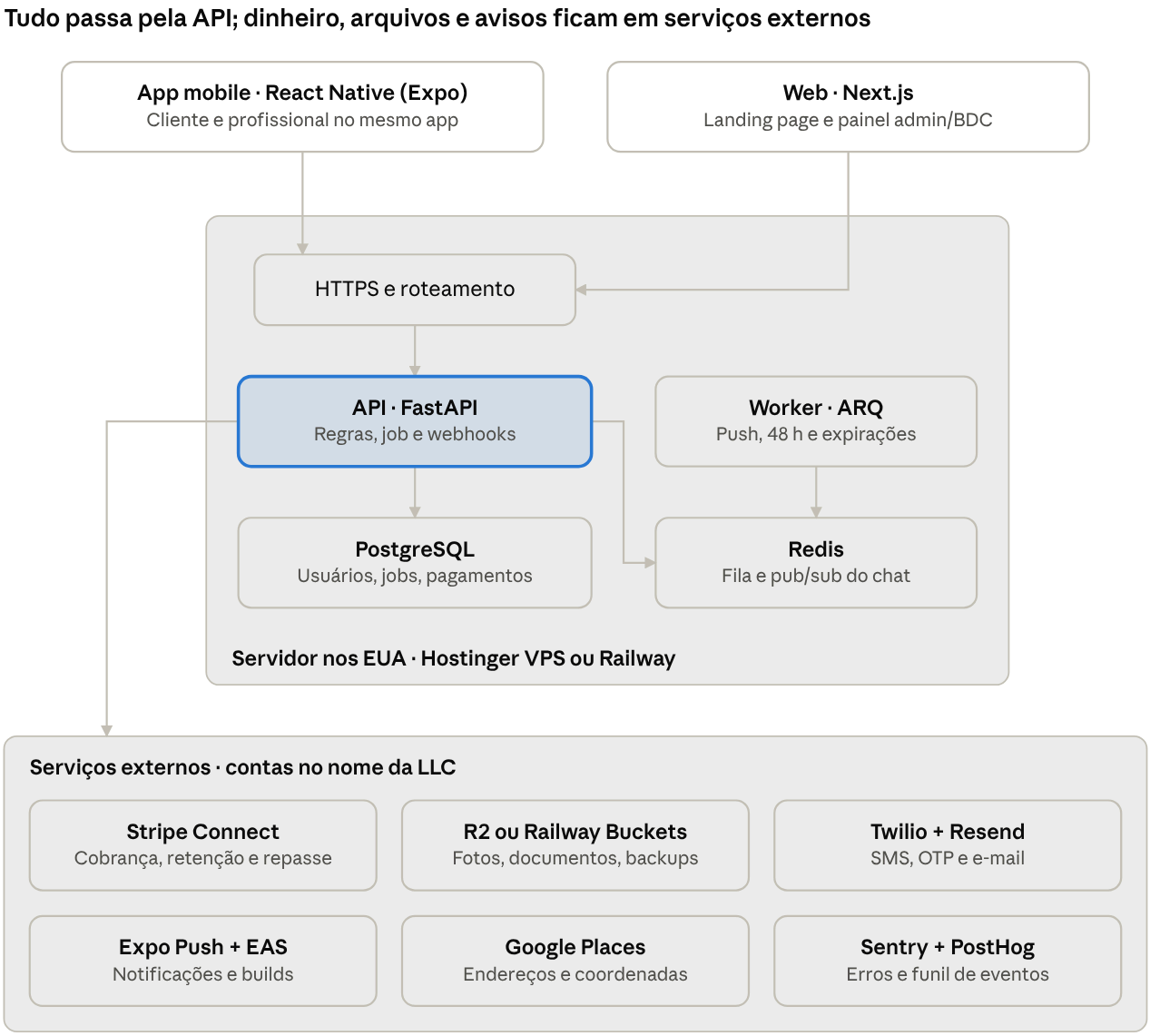
- Template reutilizável por serviço (`/house-cleaning` , `/handyman`) e por cidade + serviço (`/fort-lauderdale/handyman`), gerado das categorias e áreas cadastradas, só para combinações ativas.
- Áreas vêm do banco (região → cidades → ZIPs), sem South Florida fixo no código.

Eventos de analytics

`request_started` , `request_submitted` , `photo_uploaded` , `category_selected` , `quote_viewed` , `quote_accepted` , `appointment_scheduled` , `payment_started` , `payment_completed` , `job_completed` , `review_submitted` .

Stack e arquitetura

A stack React Native + FastAPI + Postgres serve bem e não precisa trocar: FastAPI é rápido de escrever, tipado com Pydantic, e Python já é seu terreno. O que falta são as peças em volta: pagamentos, fila de tarefas, arquivos, notificações e um painel web.



arquitetura do MVP · apps, VPS e serviços externos

App e web só falam com a API pelo Caddy; o worker cuida do que é assíncrono, e cartão, arquivos e mensagens ficam em serviços externos.

Camada	Escolha	Por quê
App mobile	React Native com Expo, Expo Router, TypeScript	Build e publicação nas lojas pelo EAS, push nativo, atualização OTA
UI do app	NativeWind (Tailwind) + componentes próprios + Reanimated	Visual caprichado, rápido e fiel à marca
Dados no app	TanStack Query + Zustand	Cache e refetch prontos; pouco estado global
Backend	FastAPI, SQLAlchemy 2 (async), Alembic, Pydantic v2	Mantém a stack; migrations versionadas
Banco	PostgreSQL 16 (PostGIS opcional)	Relacional para dinheiro; PostGIS se a área de atuação virar raio em km
Fila e agendamento	Redis + ARQ	Webhooks, notificações, auto-confirmação em 48 h, expiração de orçamentos
Pagamentos	Stripe Connect (contas Express) + Payment Sheet no app	Retenção e repasse, KYC do profissional pronto, cartão nunca passa pelo servidor
Arquivos	Cloudflare R2 ou Railway Buckets (ambos S3), URLs pré-assinadas	Fotos de pedido, antes e depois, documentos
Autenticação	JWT próprio (access + refresh) + Twilio Verify para OTP	Sem dependência de terceiros no login; telefone verificado
Notificações	Expo Push, Resend (e-mail), Twilio (SMS)	Avisos de pedido, orçamento, status e pagamento
Endereços	Google Places Autocomplete + Geocoding	Endereço validado e coordenadas para o match
Admin + landing	Next.js + shadcn/ui + TanStack Table	Um só projeto web para o BDC e a página de captação
Idiomas	i18next (app e web)	Inglês e espanhol desde o início
Observabilidade	Sentry (app, API, web), UptimeRobot, logs JSON	Erros visíveis antes do usuário reclamar

Camada	Escolha	Por quê
Infra	Hostinger (VPS + Docker Compose + Caddy) ou Railway, a decidir; GitHub Actions nos dois	Deploy por push; staging e produção separados
Backups	Hostinger: pg_dump diário para o R2 (14 dias); Railway: backups agendados + PITR	Restauração testada nos dois cenários
Analytics	PostHog	Funil pedido → orçamento → pago → concluído

Organização do código

Monorepo no GitHub da empresa:

- `apps/mobile`: React Native (cliente + profissional)
- `apps/web`: Next.js (landing + painel admin/BDC)
- `apps/api`: FastAPI + worker ARQ
- `packages/shared`: tipos TypeScript gerados do OpenAPI da API

Por que um app só para os dois lados: metade das telas é comum (login, chat, perfil, job) e publicar um app em vez de dois economiza dias de revisão nas lojas. Dá para separar depois, se o produto crescer.

Modelo de dados

O banco gira em torno de três tabelas: o pedido (`service_requests`), o orçamento aceito que vira job (`jobs`) e o pagamento (`payments`). Todo valor é guardado em centavos (inteiro), em USD.

Tabela	Campos principais	Observação
users	id, email, phone, password_hash, role (customer, pro, agent, admin), locale, status	agent e admin só acessam o painel

Tabela	Campos principais	Observação
pro_profiles	user_id, bio, foto, verification_status, stripe_account_id, payouts_enabled, rating_avg, jobs_count	Só `approved` + `payouts_enabled` recebe pedidos
pro_documents	pro_id, tipo (id, license, insurance), file_key, status, validade	Revisados pelo admin
categories	slug, nomes (en, es, pt), ícone, requires_license, reference_price, intake_questions (JSON), active	Perguntas de contexto editáveis no admin
pro_categories	pro_id, category_id	N:N
service_areas	região, cidade, zip_code, active	Expansão sem mexer no código
pro_service_areas	pro_id, service_area_id	Área de atuação do profissional
addresses	user_id, linha, cidade, zip, lat, lng	Endereço do serviço
service_requests	public_code (JF-000124), customer_id, category_id (opcional), address_id, descrição, respostas de contexto (JSON), urgência, data preferida, contato preferido, idioma, status, assigned_agent_id, next_action, first_contact_at, last_contact_at, origem (app, web, telefone), utm	Pedido do cliente; também é o item da fila do BDC
request_media	request_id, tipo (photo, video), file_key	Fotos e vídeos do pedido
request_dispatches	request_id, pro_id, sent_at, seen_at	Para quem o pedido foi enviado; mede tempo de resposta
quotes	request_id, pro_id, amount_cents, escopo, duração, data proposta, expires_at, status	Preço fechado

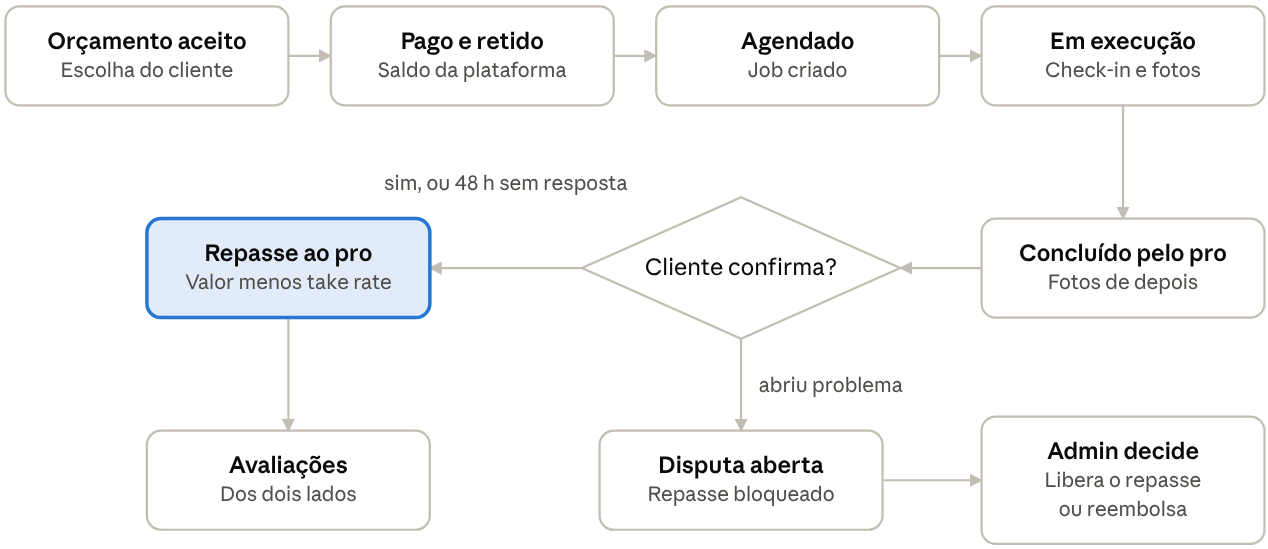
Tabela	Campos principais	Observação
jobs	request_id, quote_id, customer_id, pro_id, status, customer_price_cents, pro_payout_cents, platform_fee_cents, tax_cents, started_at, completed_at, confirmed_at	Valores congelados no pagamento
appointments	job_id, arrival_start, arrival_end, status	Separado do job para permitir reagendar
job_adjustments	job_id, tipo (charge, discount, refund), amount_cents, motivo	Cobranças extras e ajustes
job_events	job_id, tipo, ator, lat, lng, created_at	Histórico de status e check-in
job_photos	job_id, tipo (before, after), file_key	Prova do serviço
payments	job_id, stripe_payment_intent_id, amount_cents, status	Cobrança do cliente
transfers	job_id, stripe_transfer_id, amount_cents, status	Repasse ao profissional
refunds	payment_id, amount_cents, motivo, stripe_refund_id	Total ou parcial
disputes	job_id, opened_by, motivo, fotos, status, resolução	Resolvidas pelo admin
messages	thread (request ou job), sender_id, texto, redacted, read_at	Chat
internal_notes	request_id ou job_id, author_id, texto	Notas do BDC, nunca visíveis ao cliente
reviews	job_id, author_id, target_id, nota geral, comunicação, qualidade, pontualidade, custo-benefício, comentário, fotos	Uma por lado em cada job
devices	user_id, expo_push_token, plataforma	Push

Tabela	Campos principais	Observação
webhook_events	stripe_event_id, tipo, processed_at	Idempotência dos webhooks
audit_log	admin_id, ação, entidade, antes, depois	Toda ação financeira do admin fica registrada
settings	chave, valor	Take rate, janela de confirmação, N de envios

Pagamento protegido e estados do job

O cliente paga no aceite do orçamento, o dinheiro fica no saldo da plataforma e o profissional só recebe depois da confirmação. O modelo é o *separate charges and transfers* do Stripe Connect: a cobrança é da plataforma e o repasse é uma transferência para a conta Express do profissional, já sem o take rate.

O profissional só recebe depois que o cliente confirma ou o prazo de 48 h passa



estados do job · do aceite ao repasse

Enquanto o cliente não confirma, o dinheiro fica parado no saldo da plataforma; uma disputa congela o repasse até o admin decidir.

Por que não pré-autorizar o cartão: a pré-autorização expira em poucos dias, e um job pode ser agendado para semanas depois. Cobrar e reter no saldo funciona para qualquer prazo.

Passo a passo do dinheiro

1. Cliente aceita o orçamento → a API cria um PaymentIntent com o valor total.
2. O app abre o Payment Sheet do Stripe; os dados do cartão vão direto para o Stripe.
3. Webhook `payment_intent.succeeded` → o job passa para **agendado**.
4. Profissional conclui → o cliente tem 48 h para confirmar ou abrir problema.
5. Confirmado (ou 48 h sem ação) → o worker cria a transferência (valor − take rate) para a conta do profissional.
6. Problema aberto → o repasse fica bloqueado até o admin decidir: liberar, reembolsar parcial ou total.

Regras técnicas obrigatórias

- Webhooks verificados por assinatura e processados uma única vez (`webhook_events`).
- Toda cobrança, transferência e reembolso usa *idempotency key*.
- Valores sempre calculados no backend, nunca recebidos do app.
- Status só muda por transições permitidas (máquina de estados no backend).
- Stripe em modo teste até o Dia 20; modo live só no lançamento.
- Antes do Dia 11, conferir na documentação do Stripe os prazos de retenção de saldo e as regras de reembolso para a conta da LLC.

Semana 1 — Fundação (Dias 1 a 5)

Ao fim da semana, o usuário cria conta e faz login no app apontando para staging, com infra, CI/CD e design system prontos.

Dia 1 — Kickoff e contas

- [ ] Kickoff com os sócios: validar este escopo, categorias, regra de cancelamento e take rate (1 h)
- [ ] Criar as contas no nome da LLC: GitHub, Stripe (Connect), Expo/EAS, Apple Developer, Google Play Console, Cloudflare (R2 e DNS), Sentry, Twilio, Resend, Google Cloud (Places), PostHog
- [ ] Criar o monorepo com padrões (ESLint, Prettier, Ruff, pre-commit), README e convenção de commits
- [ ] Registrar decisões de arquitetura (ADRs curtos) e o glossário de status do job
- [ ] Montar o board de tasks (Linear ou GitHub Projects) com este cronograma

Entrega: contas criadas, repositório no ar, board pronto.

Dia 2 — Design

- [ ] Design system da marca: azul-marinho, laranja, branco e cinza claro, tipografia, espaçamentos, raios, ícones
- [ ] Protótipo das telas-chave no Figma: boas-vindas, home do cliente, criar pedido, orçamentos, acompanhamento do job, feed do profissional, detalhe do job, perfil, avaliação
- [ ] Definir estados vazios, de carregamento e de erro
- [ ] Validar o protótipo com o Guilherme (30 min), que cuida da marca

Entrega: protótipo navegável aprovado.

Dia 3 — Infra e backend base

- [ ] **Se Hostinger:** hardening da VPS (usuário sem root, SSH só por chave, ufw 22/80/443, fail2ban, atualizações automáticas), Docker Compose com api, worker, postgres, redis, web e caddy, e pg_dump diário para o R2 com teste de restauração
- [ ] **Se Railway:** projeto com ambientes staging e produção, serviços api, worker e web, Postgres e Redis em rede privada, backups agendados e PITR ativados, teste de restauração (cerca de meio dia)
- [ ] GitHub Actions: lint, testes e build; deploy por SSH (Hostinger) ou pela integração com git do Railway (staging na `develop`, produção na `main`)
- [ ] Esqueleto do FastAPI: config por ambiente, SQLAlchemy async, Alembic, health check, tratamento de erros, Sentry, logs JSON

Entrega: API de staging respondendo em HTTPS.

Dia 4 — Modelo de dados e autenticação

- [ ] Migrations de usuários, perfis, categorias, endereços, pedidos, orçamentos, jobs e eventos
- [ ] Cadastro e login (senha com Argon2), JWT access e refresh com rotação
- [ ] Verificação de telefone por OTP (Twilio Verify) e recuperação de senha por e-mail
- [ ] Permissões por papel (cliente, profissional, admin) e rate limit nas rotas de autenticação
- [ ] Testes de API da autenticação e seed das categorias

Entrega: autenticação completa e documentada no Swagger.

Dia 5 — Base do app

- [ ] Projeto Expo com Expo Router, TypeScript, NativeWind, fontes e tema da marca
- [ ] Componentes: botão, input, card, avatar, badge de status, bottom sheet, toast, skeleton
- [ ] Telas de boas-vindas, escolha de perfil, cadastro, OTP e login integradas à API
- [ ] Token em armazenamento seguro, refresh automático, cliente HTTP com tipos gerados do OpenAPI
- [ ] i18n inglês/espanhol e primeiro dev build no celular (EAS)

Entrega: login real no app contra staging.

Semana 2 — Profissional e pedidos (Dias 6 a 10)

Ao fim da semana, um cliente cria um pedido e recebe orçamentos de profissionais aprovados, de ponta a ponta em staging.

Dia 6 — Onboarding do profissional

- [ ] API e telas de perfil: foto, bio, anos de experiência, categorias, ZIP codes de atuação
- [ ] Upload por URL pré-assinada para o R2, com compressão da imagem no app
- [ ] Envio de documentos: documento com foto, licença (se a categoria exigir) e seguro
- [ ] Status de verificação visível no app: pendente, aprovado, recusado com motivo

Entrega: profissional completa o cadastro e fica pendente.

Dia 7 — Stripe Connect

- [ ] Criar conta Express e abrir o link de onboarding do Stripe a partir do app
- [ ] Deep link de retorno e atualização do status pelo webhook `account.updated`
- [ ] Endpoint de webhook com verificação de assinatura e idempotência, base para os próximos dias
- [ ] Aprovação de profissionais pelo SQLAdmin (painel interno automático do FastAPI), liberando o Dia 16 para a operação do BDC
- [ ] Testar o fluxo com contas de teste do Stripe

Entrega: profissional aprovado e apto a receber.

Dia 8 — Pedido do cliente

- [ ] Home do cliente com o campo "What's going on at home?" em destaque e as categorias como opção secundária
- [ ] Pedido em etapas: o que aconteceu (texto, fotos, vídeo, categoria opcional) → perguntas de contexto → local (ZIP, endereço com Google Places, data e horário) → revisão
- [ ] Perguntas de contexto lidas da categoria (JSON editável no admin), prontas para as 5 categorias do lançamento
- [ ] Checagem de ZIP atendido (Broward primeiro) e lista de espera para quem está fora
- [ ] Confirmação "Jack is on it" com número do pedido e lista "Meus pedidos" com os status do cliente

Entrega: pedido criado com texto, mídia, perguntas de contexto e endereço.

Dia 9 — Distribuição e orçamentos

- [ ] Distribuição: profissionais aprovados, da categoria, com o ZIP do pedido, ordenados por nota e tempo de resposta; envia para até N
- [ ] Feed de oportunidades no app do profissional, com detalhe e fotos do pedido
- [ ] Envio de orçamento: valor fechado, o que inclui, duração estimada, data e horário
- [ ] Expiração de orçamentos e pedidos pelo worker, com alerta ao BDC quando ninguém responde

Entrega: profissional recebe e responde pedidos.

Dia 10 — Comparação, aceite e push

- [ ] Tela de orçamentos: comparar preço, nota, jobs feitos, selo e data
- [ ] Perfil público do profissional (versão inicial)
- [ ] Push com Expo: registro de token, envio pelo worker, deep link para a tela certa
- [ ] Notificações de novo pedido (profissional), novo orçamento (cliente) e orçamento aceito (profissional)
- [ ] Demo gravada da semana para os sócios

Entrega: pedido → orçamento → aceite funcionando com push.

Semana 3 — Pagamento, execução e confiança (Dias 11 a 15)

Ao fim da semana, um job completo roda com dinheiro de teste: pagamento retido, execução, confirmação, repasse e avaliação.

Dia 11 — Cobrança

- [ ] PaymentIntent no aceite do orçamento, com valor e take rate calculados no backend
- [ ] Payment Sheet do Stripe no app (cartão, Apple Pay, Google Pay)
- [ ] Webhooks de pagamento aprovado e recusado, criando o job como agendado
- [ ] Tela de recibo e e-mail de confirmação
- [ ] Testar cartão recusado, 3D Secure e webhook duplicado

Entrega: cliente paga e o job aparece agendado para os dois lados.

Dia 12 — Execução do job

- [ ] Máquina de estados no backend, com transições válidas e histórico em `job_events`
- [ ] App do profissional: agenda, "a caminho", check-in com localização e hora, fotos de antes, concluir com fotos de depois
- [ ] App do cliente: linha do tempo do job atualizada por push e refetch
- [ ] Notificação em cada mudança de status

Entrega: job vai de agendado a concluído pelo profissional.

Dia 13 — Confirmação, repasse e cancelamento

- [ ] Tela de confirmação do cliente: confirmar ou relatar problema com fotos
- [ ] Auto-confirmação em 48 h por tarefa agendada no worker
- [ ] Transferência para a conta do profissional, com idempotência e registro em `transfers`
- [ ] Cancelamento pelo cliente ou pelo profissional, com regra de reembolso
- [ ] Tela de ganhos do profissional

Entrega: dinheiro de teste chega à conta Express do profissional.

Dia 14 — Chat

- [ ] Chat por pedido e por job: conversas, mensagens, envio de foto, não lidas
- [ ] Tempo real por WebSocket no FastAPI com Redis pub/sub, e push como reserva
- [ ] Ocultar telefone, e-mail e links no texto e avisar o usuário

- [ ] Push de nova mensagem com deep link

Entrega: cliente e profissional conversam dentro do job.

Dia 15 — Avaliações e disputas

- [ ] Avaliação dos dois lados após a confirmação: nota geral, comunicação, qualidade, pontualidade e custo-benefício, comentário e fotos opcionais, com lembrete por push
- [ ] Média e contagem recalculadas; perfil público completo com selo, avaliações e jobs feitos
- [ ] Abertura de disputa dentro da janela, bloqueando o repasse
- [ ] Endpoints de resolução para o admin: liberar, reembolso parcial ou total
- [ ] Teste ponta a ponta do ciclo completo em staging

Entrega: ciclo inteiro do job validado.

Semana 4 — Admin, landing, acabamento e lançamento (Dias 16 a 20)

Ao fim da semana, o MVP está em produção, nas lojas em teste fechado, e o BDC opera tudo pelo painel.

Dia 16 — Painel do BDC e operação

- [ ] Next.js com login por papel (agent, admin) e layout com shadcn/ui
- [ ] Fila de operações priorizada: novos, urgentes, aguardando cliente, aguardando profissional, orçamento pronto, agendados hoje, problemas e disputas
- [ ] Card do pedido com cliente, telefone, idioma, ZIP, resumo, categoria, urgência, mídia, status, agente, criação, último contato e próxima ação
- [ ] Ações do agente: classificar, pedir mais informação, atribuir profissional, mudar status, agendar follow-up e notas internas; ligar, SMS e e-mail por links `tel:`, `sms:` e `mailto:`, sem telefonia integrada
- [ ] Ações financeiras (liberar, reembolsar, resolver disputa) com confirmação e audit log, e dashboard com os KPIs do deck

Entrega: o BDC opera a fila inteira pelo painel.

Dia 17 — Landing e pedido pela web

- [ ] Landing em Next.js, renderizada no servidor, seguindo a seção de landing deste doc
- [ ] Pedido pela web sem login, com o mesmo fluxo do app, criando o pedido na fila do BDC com UTM
- [ ] Templates de página por serviço e por cidade + serviço, com metadados de SEO, só para as categorias ativas
- [ ] Página "Para profissionais" com link para baixar o app
- [ ] Eventos de analytics no PostHog, na landing e no app

Entrega: tráfego pago já pode apontar para a landing.

Dia 18 — Acabamento visual

- [ ] Revisão tela a tela contra o protótipo: espaçamento, tipografia, contraste
- [ ] Skeletons, estados vazios ilustrados, mensagens de erro claras, microanimações
- [ ] Ícone do app, splash screen, screenshots e textos das lojas
- [ ] Templates de e-mail e SMS com a marca
- [ ] Revisão das traduções em espanhol por um falante nativo

Entrega: app com cara de produto pronto.

Dia 19 — QA e segurança

- [ ] Roteiro de testes manuais de todos os fluxos em iOS e Android, em aparelho real
- [ ] Testes automatizados das regras críticas: cálculo de valores, transições de status, webhooks, permissões
- [ ] Revisão de segurança: rate limit, validação de entrada, arquivos só por URL assinada, segredos fora do repositório, CORS
- [ ] Termos de uso, privacidade e contrato do profissional no app; exclusão de conta (exigida pela Apple)
- [ ] Correção dos bugs encontrados

Entrega: nenhum bug crítico aberto.

Dia 20 — Lançamento

- [ ] Deploy em produção, migrations, seed de categorias e configurações

- [ ] Stripe em modo live, webhooks de produção e um pagamento real de valor baixo com reembolso
- [ ] Build de produção no EAS e envio para App Store (TestFlight) e Google Play (teste fechado)
- [ ] Runbook: deploy, restaurar backup, reprocessar webhook, resolver disputa
- [ ] Demo final para os sócios e treinamento do BDC no painel

Entrega: MVP em produção.

Folga até o dia 30

Os dias corridos restantes cobrem a revisão das lojas (a Apple pode pedir ajustes), correções do primeiro uso real e o cadastro dos primeiros profissionais.

Como adiantar o prazo

- **Landing com o Guilherme:** ele cuida de marketing; se ele montar a landing a partir do README, o Dia 17 fica só com o pedido pela web e os templates de SEO.
- **Admin mínimo antes:** o SQLAdmin já aprova profissionais desde o Dia 7; usá-lo também para ver jobs e adiar essa tela no Next.js. Economiza cerca de meio dia.
- **Chat sem WebSocket:** polling curto + push. Economiza cerca de meio dia.
- **Menos Figma:** construir direto com o design system e revisar no app. Economiza meio dia, com algum risco de retrabalho.
- **Um idioma no lançamento:** inglês primeiro, espanhol na semana seguinte.

Com esses cortes, o MVP sai em cerca de 17 a 18 dias úteis.

Definição de pronto

O MVP está pronto quando um cliente real contrata, paga e avalia um profissional real sem ajuda técnica, e o BDC opera tudo pelo painel.

Critérios de aceite do lançamento

- [ ] Cliente cria pedido, recebe orçamento, paga, acompanha, confirma e avalia, em iOS e Android
- [ ] Profissional se cadastra, envia documentos, conecta o Stripe, é aprovado, orça, executa e recebe o repasse
- [ ] Pagamento fica retido até a confirmação, e a auto-confirmação em 48 h funciona

- [ ] Disputa bloqueia o repasse e o admin consegue reembolsar total ou parcialmente
- [ ] Push, e-mail e SMS chegam nos eventos principais
- [ ] Painel mostra a fila de leads com cronômetro e os KPIs do deck
- [ ] App em inglês e espanhol
- [ ] Nenhum erro crítico aberto no Sentry; backup diário rodando e restauração testada
- [ ] Termos, privacidade e exclusão de conta disponíveis
- [ ] App aprovado, ou em teste fechado, nas duas lojas

Padrão de cada task

Uma task só fecha com código revisado, migration quando houver, teste das regras afetadas, deploy em staging e verificação no aparelho.

Riscos, premissas e dependências

O maior risco ao prazo não é o código, e sim as aprovações externas (Apple, lojas e Stripe), por isso todas começam no Dia 1.

Risco	Impacto no prazo	Mitigação
Conta Apple Developer como empresa exige D-U-N-S e pode demorar	Trava a publicação no iOS	Pedir no Dia 1; se atrasar, lançar Android primeiro e iOS assim que liberar
Revisão das lojas rejeita o app	1 a 5 dias	Exclusão de conta, termos e login de teste para o revisor; usar a folga
Verificação da conta Stripe da LLC	Bloqueia o modo live	Abrir no Dia 1 com os dados da LLC
Escopo cresce durante o mês	Estoura os 30 dias	Pedido novo vai para o roadmap; só entra trocando por algo do mesmo tamanho
Dev solo: doença ou imprevisto	Atraso direto	Folga de cerca de uma semana; prioridade é o ciclo do job
Textos legais e marca atrasam	Trava os Dias 19 e 20	Pedir aos sócios até o Dia 10

Risco	Impacto no prazo	Mitigação
Desintermediação depois do primeiro job	Receita	Pagamento e garantia só pelo app, contatos ocultos no chat, reputação como incentivo
Servidor único fora do ar (mais crítico na Hostinger)	Webhooks atrasados	Stripe reenvia webhooks; alerta no UptimeRobot; setup reproduzível em Compose

O que preciso dos sócios

- [ ] Contas criadas ou acesso concedido no nome da LLC (Dia 1)
- [ ] Logo, cores e arquivos da marca (Dia 2)
- [ ] Take rate, regra de cancelamento e prazo de garantia (Dia 1)
- [ ] Lista de ZIP codes atendidos no lançamento (Dia 8)
- [ ] Termos de uso, privacidade e contrato do profissional (Dia 10)
- [ ] Uma pessoa do BDC para testar o painel (Dia 17)
- [ ] Profissionais reais para teste beta (Dia 18)

Pós-MVP e pendências

Depois do lançamento, a prioridade é o que aumenta confiança e ticket médio, na ordem que o deck propõe.

Fase	Funcionalidades
v1.1 (mês 2)	Rastreamento do profissional no mapa, preço tabelado para limpeza (por quartos e banheiros), reagendamento, gorjeta
v1.2 (meses 3 e 4)	Background check com Checkr, pagamento por marcos em jobs grandes, cupons e indicação, categorias de alto valor (HVAC, elétrica, encanamento, telhado)

Fase	Funcionalidades
v2 (meses 4 a 6)	Classificação do pedido e entrada por voz com IA, análise das conversas do BDC, planos recorrentes, "My Home" (histórico da casa), contas B2B (property managers, hosts de Airbnb), apps separados se fizer sentido

Para decidir com os sócios antes do Dia 1

- [ ] Hospedagem: Hostinger ou Railway (comparação no doc de custos)
- [ ] Take rate final e quem absorve as taxas do Stripe (cliente, profissional ou plataforma)
- [ ] Regra de cancelamento e reembolso
- [ ] Prazo da janela de garantia e texto jurídico aprovado para ela
- [ ] Quem testa e aprova cada entrega semanal
- [ ] Quem escreve os textos legais e o conteúdo da landing
- [ ] Português entra no app ou só no atendimento do BDC?